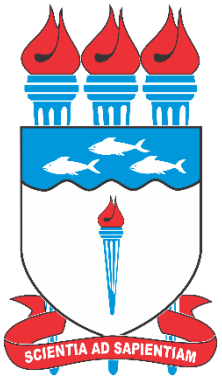
UNIVERSIDADE FEDERAL DE ALAGOAS
INSTITUTO DE COMPUTAÇÃO
CURSO DE CIÊNCIA DA COMPUTAÇÃO



DESENVOLVIMENTO DE UM COMPILADOR PARA A LINGUAGEM MINIPAR 2026.2 ORIENTADA A OBJETOS UTILIZANDO REAL PARALELISMO COM EXECUÇÃO EM COMPUTADORES DIFERENTES

Uso de Fork e Join e Implementação usando os Canais de Comunicação (c_channel) do Minipar

ARTUR FERREIRA MARQUES DA SILVA
JOÃO MANOEL SANTOS LIMA

Repositório do código-fonte:
https://github.com/arturfmarques/minipar-2026.2

MACEIÓ – AL
2026

ARTUR FERREIRA MARQUES DA SILVA
JOÃO MANOEL SANTOS LIMA

DESENVOLVIMENTO DE UM COMPILADOR PARA A LINGUAGEM MINIPAR 2026.2 ORIENTADA A OBJETOS UTILIZANDO REAL PARALELISMO COM EXECUÇÃO EM COMPUTADORES DIFERENTES

Relatório técnico apresentado à disciplina de Compiladores do Curso de Ciência da Computação do Instituto de Computação da Universidade Federal de Alagoas, período 2026.2, como requisito parcial de avaliação do projeto do compilador MiniPar 2026.2.

Professor: Arturo Hernandez Dominguez

Integrantes:
Artur Ferreira Marques da Silva – matrícula 202437820
João Manoel Santos Lima – matrícula 202407474

Repositório:
https://github.com/arturfmarques/MiniPar-2026.2

MACEIÓ – AL
2026

RESUMO

Este relatório descreve o compilador da linguagem MiniPar 2026.2, desenvolvido em Python a partir da versão 2026.1. O compilador realiza análise léxica, sintática e semântica, mantém a tabela de símbolos, gera código de três endereços (TAC) e gera C++20, compilado pelo G++ com -O3 em um executável. A linguagem é orientada a objetos e oferece SEQ, PAR, Fork e Join com execução local, em processador lógico escolhido ou em outro processo via TCP, além de canais de comunicação. O projeto mantém as interfaces de terminal e Web e acrescenta uma interface Web educacional. Os onze programas de teste foram implementados em MiniPar, e os registros de validação informam 519 testes automatizados aprovados.

Palavras-chave: compiladores; MiniPar; código de três endereços; paralelismo; Fork e Join; computação distribuída; orientação a objetos.

SUMÁRIO

1 INTRODUÇÃO 5
2 A LINGUAGEM MINIPAR 2026.2 E SUA GRAMÁTICA 5
3 ARQUITETURA DO COMPILADOR 6
4 ANALISADOR LÉXICO 7
5 ANALISADOR SINTÁTICO E AST 8
6 ANÁLISE SEMÂNTICA E TRATAMENTO DE ERROS 8
7 TABELA DE SÍMBOLOS 9
8 GERAÇÃO DE TAC E DE CÓDIGO C++ 10
9 ORIENTAÇÃO A OBJETOS, PARALELISMO E EXECUÇÃO DISTRIBUÍDA 11
10 INTERFACES DO COMPILADOR 12
11 PRODUCT BACKLOG E SPRINT BACKLOGS 16
12 MODELAGEM UML 17
13 TESTES DE INTEGRAÇÃO 20
14 TESTES DOS PROGRAMAS DE EXEMPLO 21
14.1 Teste 1 — Calculadora cliente/servidor 21
14.2 Teste 2 — Fatorial e Fibonacci em paralelo 22
14.3 Teste 3 — Neurônio 23
14.4 Teste 4 — Rede neural XOR 24
14.5 Teste 5 — Sistema de recomendação 24
14.6 Teste 6 — Quicksort recursivo com classes 26
14.7 Teste 7 — Quicksort iterativo 26
14.8 Testes 8, 9 e 10 — Avaliação de atividades e fractais 26
14.9 Teste 11 — Minicompilador de expressões para ARMv7 29
15 IMPLEMENTAÇÃO 33
16 CONCLUSÃO 34
REFERÊNCIAS 35

1 INTRODUÇÃO

Este relatório apresenta o compilador da linguagem MiniPar 2026.2, desenvolvido na disciplina de Compiladores da UFAL. O compilador, escrito em Python e organizado em componentes, realiza análise léxica, sintática e semântica, mantém a tabela de símbolos, gera código intermediário de três endereços (TAC) e gera código C++20, compilado pelo G++ com `-O3` em um executável nativo.

O projeto foi desenvolvido a partir do compilador MiniPar 2026.1 (projeto MiniPar-Improvement), cuja base técnica foi reutilizada e adaptada às exigências de 2026.2. A versão 2026.2 acrescentou Fork e Join com destino local ou remoto e seleção opcional de processador lógico, um runtime C++ com tarefas, canais TCP e chamadas remotas, a análise semântica de isolamento das tarefas, o TAC de tarefas, os programas de teste 1 a 11 e uma segunda interface Web, educacional.

Objetivo geral: implementar um compilador para o MiniPar 2026.2, orientado a objetos e com paralelismo real, que gere TAC e um executável nativo. **Objetivos específicos:** ampliar a BNF 2026.1; implementar os analisadores e a tabela de símbolos; gerar TAC e C++; suportar classes, herança e recursão; executar SEQ, PAR, Fork e Join com threads reais e comunicação por mensagens; executar Forks em outro processo por TCP; manter as interfaces de terminal e Web e criar a interface educacional; e validar os onze programas de teste.

2 A LINGUAGEM MINIPAR 2026.2 E SUA GRAMÁTICA

Um programa MiniPar é uma sequência de classes, funções e comandos, normalmente agrupados em `seq { ... }` (execução sequencial) ou `par { ... }` (cada filho é uma tarefa em thread própria). Os tipos são `number` (double), `int` (inteiro de 64 bits com verificação de overflow), `bool`, `string`, `list`, `matrix`, `c_channel`, `task`, `void` e nomes de classe. Há `if/else`, `while`, `do-while`, `for` clássico e `for-in`, funções recursivas, classes com atributos, métodos, `init` (construtor), `this` e herança (`extends`), `input`, `print` e comentários com `#`. Canais são criados com `c_channel(host, porta)` e usados com `send` e `receive`, sempre por TCP. Fork cria uma subtarefa que executa uma função global; Join aguarda a subtarefa e devolve seu resultado.

```
func square(number value) -> number { return value * value; }
seq {
    var a = fork local square(7);                    # thread local
    var b = fork remote("127.0.0.1", 7000) square(8); # trabalhador TCP
    var c = fork local square(9) on core(1);         # processador lógico 1
    print(join(a), join(b), join(c));
}
```

A gramática oficial está em `BNF-grammar.md`. Ela parte da BNF 2026.1, como exige o enunciado, e acrescenta: as palavras reservadas `fork`, `join`, `local`, `remote` e `task`; as produções de Fork, Join e destino da tarefa; o sufixo contextual `on core(n)`; tipos anotados de parâmetros e retorno (`-> tipo`); atributos tipados em classes; e correções no `for`. A precedência dos operadores é expressa por uma cadeia de não terminais (Observação 2 do enunciado). Principais produções:

Quadro 1 – Produções principais da BNF do MiniPar 2026.2 (resumo de BNF-grammar.md)

```
<programa>    ::= { <item> } EOF
<item>        ::= <decl_classe> | <decl_funcao> | <comando>
<decl_classe> ::= "class" ID [ "extends" ID ] "{" { <decl_var> | <decl_funcao> } "}"
<decl_funcao> ::= "func" ID "(" [ <param> { "," <param> } ] ")"
                  [ "->" <tipo> ] <bloco_seq>
<param>       ::= [ <tipo> ] ID
<decl_var>    ::= ( "var" | <tipo> ) ID [ "=" <expr> ] ";"
<tipo>        ::= "number" | "int" | "string" | "bool" | "void" | "list" | "dict"
                | "task" | "c_channel" | "matrix" | ID
<comando>     ::= <decl_var> | <atribuicao> | <bloco_seq> | <bloco_par>
                | "if" "(" <expr> ")" <item> [ "else" <item> ]
                | "while" "(" <expr> ")" <item>
                | "do" <item> "while" "(" <expr> ")"
                | "for" "(" <for_cabecalho> ")" <item>
                | "print" "(" [ <args> ] ")" ";" | "send" "(" <expr> "," <expr> ")" ";"
                | "return" [ <expr> ] ";" | "break" ";" | "continue" ";" | <chamada> ";"
<bloco seq>   ::= [ "seq" ] "{" { <item> } "}"
<bloco_par>   ::= "par" "{" { <item> } "}"
<atribuicao>  ::= ( ID | <posfixo> "." ID | <posfixo> "[" <expr> "]" ) "=" <expr> ";"
<expr>        ::= <e_logico> { "||" <e_logico> }
<e_logico>    ::= <comparacao> { "&&" <comparacao> }
<comparacao>  ::= <termo> { ( "==" | "!=" | "<" | ">" | "<=" | ">=" ) <termo> }
<termo>       ::= <fator> { ( "+" | "-" ) <fator> }
<fator>       ::= <unario> { ( "*" | "/" | "%" ) <unario> }
<unario>      ::= ( "+" | "-" | "!" ) <unario> | <posfixo>
<posfixo>     ::= <primario> { "." ID [ "(" [ <args> ] ")" ] | "[" <expr> "]" }
<primario>    ::= NUMBER | STRING | "true" | "false" | "this"
                | ID [ "(" [ <args> ] ")" ]
                | "new" ID "(" [ <args> ] ")" | "[" [ <args> ] "]"
                | "matrix" "(" <expr> "," <expr> "," <expr> ")"
                | "c_channel" "(" <expr> "," <expr> ")" | "receive" "(" <expr> ")"
                | "input" "(" [ <expr> ] ")" | <fork> | <join> | "(" <expr> ")"
<fork>        ::= "fork" <destino> ID "(" [ <args> ] ")" [ "on" "core" "(" <expr> ")" ]
<destino>     ::= "local" | "remote" "(" <expr> "," <expr> ")"
<join>        ::= "join" "(" ID ")"
```

Fonte: elaborado pelos autores (2026).

3 ARQUITETURA DO COMPILADOR

O fluxo é: código MiniPar → Lexer → Parser → AST → análise semântica e tabela de símbolos → TAC e C++ → G++ → executável. O TAC e o C++ são gerados em paralelo a partir da mesma AST validada; o gerador C++ não lê o arquivo TAC. A função `compile_file`

de `main.py` encadeia as fases e grava os artefatos em `output/`; a opção `--tac-only` gera apenas o TAC. O Quadro 2 apresenta os componentes, e o diagrama de componentes está no Capítulo 12.

Quadro 2 – Componentes do compilador

Componente	**Arquivo**	**Responsabilidade**
Lexer	`src/lexer.py`	Gera a lista de tokens com tipo, lexema, linha e coluna
Parser e AST	`src/parser.py`, `src/ast_nodes.py`	Descida recursiva; constrói a AST (38 classes de nós)
Analisador semântico e tabela de símbolos	`src/semantic.py`	Escopos, tipos, chamadas, classes, regras de tarefas
Diagnósticos	`src/diagnostics.py`	Erros com fase, mensagem, linha e coluna
Gerador TAC	`src/tac.py`	Código de três endereços em `output/<nome>.tac`
Gerador C++ e runtime	`src/codegen.py`, `src/cpp_template.py`, `src/runtime/*.hpp`	C++20 com runtime de tarefas, canais, RPC e afinidade
Toolchain	`src/toolchain.py`	Executa `g++ -std=c++20 -O3 -pthread` (e `-lws2_32` no Windows)

Fonte: elaborado pelos autores (2026).

4 ANALISADOR LÉXICO

A classe `Lexer` percorre o texto caractere a caractere. Letras e _ iniciam identificadores, consultados no dicionário `KEYWORDS` para reconhecer palavras reservadas; dígitos formam números; aspas formam strings com escapes; os demais caracteres formam operadores (`==`, `!=`, `<=`, `>=`, `&&`, `||` e `->` olhando o caractere seguinte) e delimitadores. Espaços e comentários `#` (até o fim da linha) são descartados. Cada token guarda linha e coluna. String não terminada e caractere inválido geram `LexerError` com a posição.

Quadro 3 – Pseudocódigo do analisador léxico (src/lexer.py)

```
ALGORITMO Lexer.tokenize()
  tokens ← []
  ENQUANTO houver caracteres:
      pular espaços e comentários ('#' até o fim da linha)
      lin, col ← posição atual;  c ← peek()
      SE c é letra ou '_':  lexema ← letras/dígitos/'_'
                            adiciona Token(KEYWORDS[lexema] ou IDENTIFIER, ...)
      SENÃO SE c é dígito:  lexema ← dígitos ['.' dígitos]; adiciona Token(NUMBER, ...)
      SENÃO SE c = '"':     lê até '"' decodificando \n \t \" \\
                            SE fim do texto ENTÃO erro "string nao terminada"
                            adiciona Token(STRING_LITERAL, ...)
      SENÃO:                op ← operador de 2 caracteres, se houver, senão de 1
                            adiciona token do operador ou Token(UNKNOWN, ...)
  SE existe token UNKNOWN ENTÃO LANÇA LexerError("caractere invalido", lin, col)
  adiciona Token(END_OF_FILE);  RETORNA tokens
```

Fonte: elaborado pelos autores (2026).

5 ANALISADOR SINTÁTICO E AST

A classe `Parser` usa descida recursiva: cada construção tem um método (`_parse_class_decl`, `_parse_func_decl`, `_parse_par_block`, `_parse_for_stmt`, `_parse_fork` etc.). A precedência segue a cadeia `_parse_or` → `_parse_and` → `_parse_comparison` → `_parse_term` → `_parse_factor` → `_parse_unary` → `_parse_primary`, com associatividade à esquerda. O resultado é uma AST de dataclasses: `Program`, blocos (`SeqBlock`, `ParBlock`), declarações (`VarDecl`, `FuncDecl`, `ClassDecl`), comandos (`IfStmt`, `WhileStmt`, `ForStmt`, `ReturnStmt`...), expressões (`BinaryExpr`, `MethodCall`, `NewExpr`...) e os nós de 2026.2 `ForkExpr`, `TaskDestination` e `JoinExpr`. Erros geram `ParseError` com linha, coluna e o token encontrado.

Quadro 4 – Pseudocódigo do analisador sintático (src/parser.py)

```
ALGORITMO Parser.parse()
  programa ← Program();  ENQUANTO token ≠ EOF: programa.adiciona(parse_statement())

FUNÇÃO parse_statement()
  CASO token: class → parse_class_decl();  func → parse_func_decl()
              var ou tipo seguido de nome → parse_var_decl()
              seq/"{" → bloco SEQ;  par → bloco PAR
              if/while/do/for/print/send/return → método específico
              outro → e ← parse_expression()
                      SE "=" ENTÃO RETORNA atribuição(e, parse_expression())
                      SE e é chamada ou join ENTÃO exige ";"; RETORNA e
                      SENÃO erro "Instrucao ou expressao isolada invalida"

FUNÇÃO nível_binário(próximo, operadores)          // or, and, comparação, termo, fator
  e ← próximo();  ENQUANTO token ∈ operadores: e ← BinaryExpr(op, e, próximo())
  RETORNA e

FUNÇÃO parse_primary()
  nó ← literal | identificador | chamada | new | this | lista | matrix | c_channel
       | receive | input | parse_fork() | join(ID) | "(" expressão ")"
  ENQUANTO "." ou "[": nó ← PropertyAccess, MethodCall ou IndexExpr
  RETORNA nó

FUNÇÃO parse_fork()
  destino ← "local" | "remote" "(" host "," porta ")";  função ← ID;  args ← "(" ... ")"
  SE próximo é "on" "core" ENTÃO destino.core ← expressão entre parênteses
  RETORNA ForkExpr(função, args, destino)
```

Fonte: elaborado pelos autores (2026).

6 ANÁLISE SEMÂNTICA E TRATAMENTO DE ERROS

A classe `SemanticAnalyzer` trabalha em passadas: registra todas as funções e classes; infere por ponto fixo os retornos não declarados (permitindo recursão e chamadas a funções definidas depois); valida cada nó; e, por fim, verifica o isolamento das tarefas. São verificados: variáveis declaradas e redeclaradas, compatibilidade de tipos, aridade de chamadas, existência de atributos e métodos (com herança), `init` e `new`, `this`, `return`, `break` e `continue` em contexto válido, canais e portas, e as regras de Fork e Join. Para atender à exigência de tarefas

independentes, uma tarefa de PAR não pode escrever em variáveis externas, e a função executada por Fork (e as que ela chama) não pode ler variáveis globais.

Lexer e Parser param no primeiro erro; o analisador semântico acumula todos os erros e impede a geração de TAC e C++. Os diagnósticos seguem o formato [ERRO FASE] Linha L, coluna C: mensagem, por exemplo [ERRO SEMANTICO] Linha 3, coluna 3: Variavel 'y' nao declarada. Erros de execução (divisão por zero, índice inválido, overflow de int, prazo de canal) são lançados pelo código gerado e exibidos como [ERRO RUNTIME]. Comentários # são tratados pelo lexer.

Quadro 5 – Pseudocódigo do analisador semântico (src/semantic.py)

```
ALGORITMO SemanticAnalyzer.analyze(programa)
  registra funções e classes (atributos, métodos, superclasse)
  REPITA até estabilizar: infere retornos não declarados
  PARA CADA item: validate_node(item)
  PARA CADA alvo de Fork: SE alguma função alcançável lê variável global
                          ENTÃO erro "Tarefa independente captura estado global"
  RETORNA não houve erros

PROCEDIMENTO validate_node(nó)
  VarDecl:     valida valor; verifica tipo; define símbolo (erro se redeclarado)
  Assignment:  variável existe; tipos compatíveis; sem escrita externa em tarefa
  Seq/If/Laço: abre escopo; valida filhos; fecha escopo
  ParBlock:    cada filho em escopo próprio marcado como fronteira de tarefa
  FuncDecl:    escopo dos parâmetros; retorno em todos os caminhos se declarado
  ClassDecl:   superclasse existente; valida atributos e métodos
  Chamada:     função/método existente (busca na herança); aridade e tipos
  ForkExpr:    inicializa variável; alvo é função global; on core: local, inteiro ≥ 0
  JoinExpr:    variável deve ter tipo task<T>
  Identificador: deve estar declarado
  erro → Diagnostic("SEMANTICO", mensagem, linha, coluna) adicionado à lista
```

Fonte: elaborado pelos autores (2026).

7 TABELA DE SÍMBOLOS

A tabela de símbolos é formada pelas estruturas do `SemanticAnalyzer`: uma pilha de escopos `_scopes` (dicionários nome → `Symbol`, sendo o índice 0 o global), e os dicionários `_functions` e `_classes`. Cada `Symbol` guarda nome, tipo, categoria (variável, parâmetro, função ou classe), parâmetros e seus tipos, tipo de retorno, atributos, métodos e superclasse. Escopos são abertos por blocos, ramos, laços, funções, classes e por cada tarefa de PAR. A busca vai do escopo mais interno ao global; membros de classes são procurados também nos ancestrais. Para a Web educacional, os escopos fechados são guardados em `scope_history` e exibidos como tabela.

Quadro 6 – Pseudocódigo da tabela de símbolos

```
escopos ← [ {} ]   // global
PROCEDIMENTO entrar_escopo(): escopos.empilha({})
PROCEDIMENTO sair_escopo():   guarda cópia em scope_history; escopos.desempilha()
PROCEDIMENTO definir(nome, tipo, linha)
  SE nome ∈ topo ENTÃO erro "Variavel ja declarada"
  SENÃO topo[nome] ← Symbol(nome, tipo, categoria, linha)
FUNÇÃO buscar(nome)
  PARA escopo DO topo AO global: SE nome ∈ escopo ENTÃO RETORNA escopo[nome]
  RETORNA nulo
FUNÇÃO membro(classe, nome)
  ENQUANTO classe existe: SE nome ∈ classe.atributos/métodos ENTÃO RETORNA
                          classe ← classe.superclasse
  RETORNA nulo
```

Fonte: elaborado pelos autores (2026).

8 GERAÇÃO DE TAC E DE CÓDIGO C++

TAC. A classe `TACGenerator` percorre a AST validada e emite instruções com no máximo uma operação, usando temporários (`t0`, `t1`...) e rótulos (`WHILE_0`, `ENDIF_2`...). Variáveis de escopos internos recebem sufixo (`pivot@S4`). `&&` e `||` geram curto-circuito; PAR gera `spawn` de cada tarefa e `join` na barreira; Fork gera `task_param_copy`, `t = fork local|remote(h,p) [on core(c)], f, n` e `r = join t`. O TAC é gerado da AST de qualquer programa, sem textos prontos. Trecho real do Teste 7 (Quicksort iterativo):

Quadro 7 – TAC gerado para o Teste 7 (trecho)

```
WHILE_0:
t5 = call len, 1
t6 = t5 > 0
ifFalse t6 goto ENDWHILE_1
t7 = ref stack@S2
t8 = callmethod t7, pop, 0
fim@S4 = t8
...
t12 = arr[fim@S4]
pivot@S4 = t12
WHILE 3:
t14 = j@S4 < fim@S4
ifFalse t14 goto ENDWHILE_4
...
t19 = index_addr arr, i@S4
store t19, t20
...
goto WHILE_0
ENDWHILE_1:
return arr
```

Fonte: elaborado pelos autores (2026).

C++. O compilador é escrito em Python, mas a linguagem-alvo é C++20. A classe `CppCodeGenerator` traduz a AST: classes viram classes C++ manipuladas por `std::shared_ptr`, `new C(args)` vira uma fábrica que chama `init`, `par` vira um grupo de tarefas, `fork local` vira `mp::fork_local`, `fork remote` vira `mp::fork_remote` e `join` vira `mp::join`. O arquivo gerado incorpora o runtime de `src/runtime/` e um `main` que

também atende o modo trabalhador (`--worker`). Em seguida, `src/toolchain.py` executa `g++ -std=c++20 -O3 arquivo.cpp -o arquivo -pthread` (mais `-lws2_32` no Windows), gerando o executável.

9 ORIENTAÇÃO A OBJETOS, PARALELISMO E EXECUÇÃO DISTRIBUÍDA

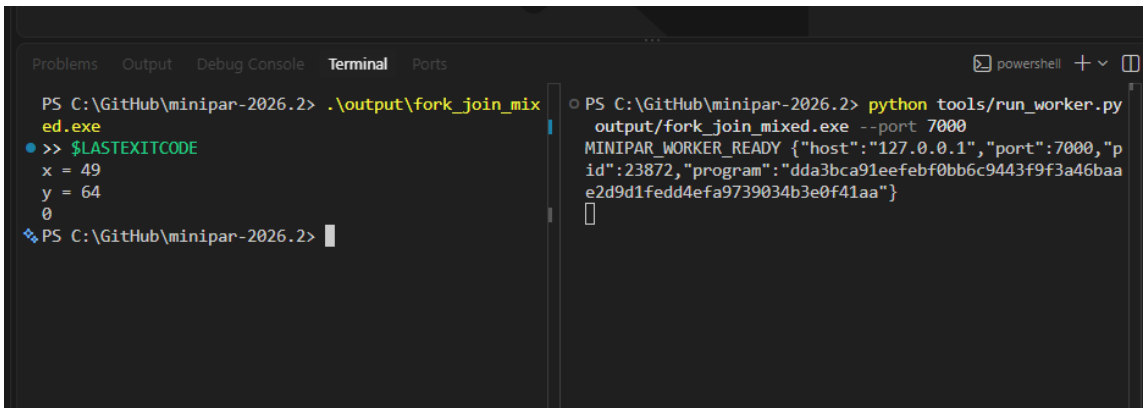
Orientação a objetos. Classes têm atributos, métodos, construtor `init`, `this` e herança; métodos podem ser recursivos. São usadas nos Testes 3 (`Neuronio`), 4 (`RedeNeuralXOR`), 5 (cinco classes), 6 (`Quicksort`) e 11 (onze classes). O Teste 7 usa somente funções, como pede o enunciado.

SEQ, PAR, Fork e Join. Cada filho de PAR e cada Fork local executa em uma thread real (`std::async`). As tarefas recebem cópias dos dados que leem (snapshots) e não compartilham estado; a troca de dados é feita por canais. Join espera a tarefa e devolve uma cópia do resultado ou propaga o erro.

Processadores. `fork local f(x) on core(n)` fixa a tarefa no processador lógico `n` antes de executar a função e confere a CPU usada (Windows: `SetThreadGroupAffinity`; Linux: `pthread_setaffinity_np`). Processadores lógicos não são necessariamente núcleos físicos distintos; a topologia física não foi verificada.

Canais e computadores diferentes. `send` e `receive` usam TCP com quadros e confirmação. `fork remote(host, porta)` envia a função e os argumentos a um trabalhador, que é o mesmo executável iniciado com `--worker`; ele só executa funções registradas pelo compilador para aquele programa e devolve o resultado ou o erro ao Join. Esses recursos foram validados entre processos distintos no mesmo computador (`127.0.0.1`); a execução em dois computadores físicos ainda não foi realizada.

Figura 1 – Fork local e remoto com trabalhador TCP



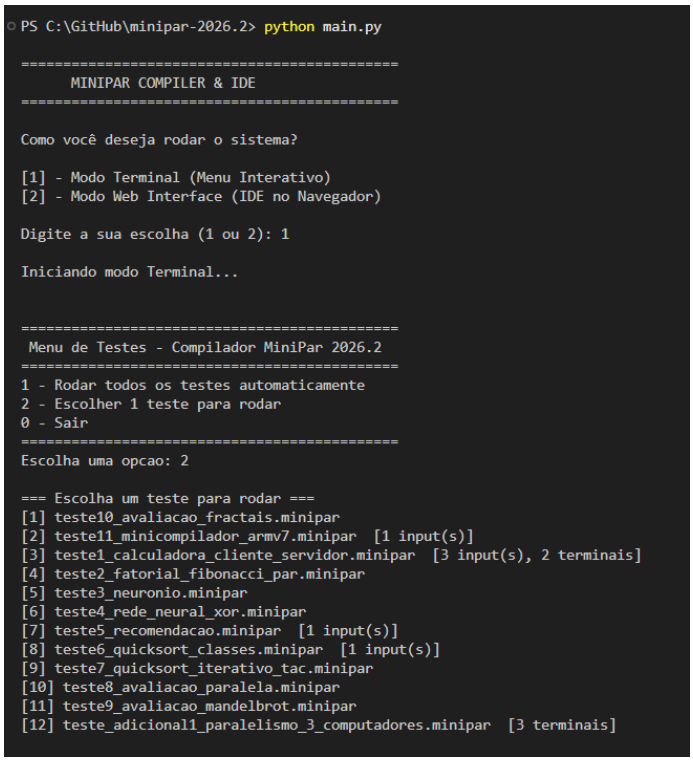
Fonte: captura de tela do sistema MiniPar 2026.2, elaborada pelos autores (2026).

10 INTERFACES DO COMPILADOR

As três interfaces usam o mesmo compilador. As duas interfaces Web rodam no mesmo servidor Flask (`python app.py`, porta 5000), feitas em HTML, CSS e JavaScript com o editor CodeMirror, e um cabeçalho comum alterna entre elas preservando o código.

Interface 1 — Terminal. `python main.py` oferece o modo terminal ou o Web. O menu (`menu.py`) lista os programas de `tests/`, compila o escolhido mostrando as quatro fases e executa o binário.

Figura 2 – Interface de terminal



```
PS C:\GitHub\minipar-2026.2> python main.py
[12] teste_adicional1_paralelismo_3_computadores.minipar  [3 terminais]
[13] teste_adicional2_tapete_sierpinski.minipar
[14] teste_ascii_art_divertidapkrl.minipar
[15] teste_bnf_controles_input.minipar  [1 input(s)]

Digite o numero do teste: 4

  Programa: teste2_fatorial_fibonacci_par

=== Compilando: tests\teste2_fatorial_fibonacci_par.minipar ===
[Fase 1] Analise Lexica e Sintatica...
[Fase 2] Analise Semantica e Tabela de Simbolos...
[Fase 3] Geracao de Codigo Intermediario (TAC - 3 enderecos)...
[TAC] Arquivo gerado: output\teste2_fatorial_fibonacci_par.tac
[Fase 4] Transpilacao e Chamada do GCC (-O3)...
[CODEGEN] Compilando via g++ com -std=c++20 e -O3...
[CODEGEN] Compilador: C:\msys64\mingw64\bin\g++.exe
[SUCESSO] Executavel: output\teste2_fatorial_fibonacci_par.exe

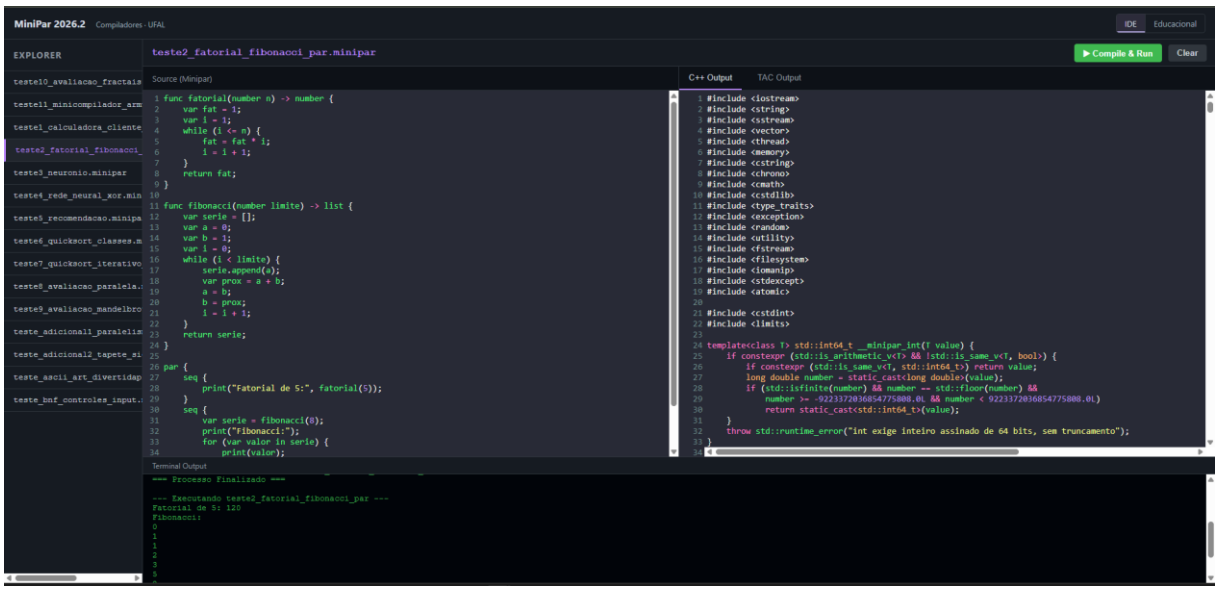
--- Executando teste2_fatorial_fibonacci_par ---
Fatorial de 5: 120
Fibonacci:
0
1
1
2
3
5
8
13

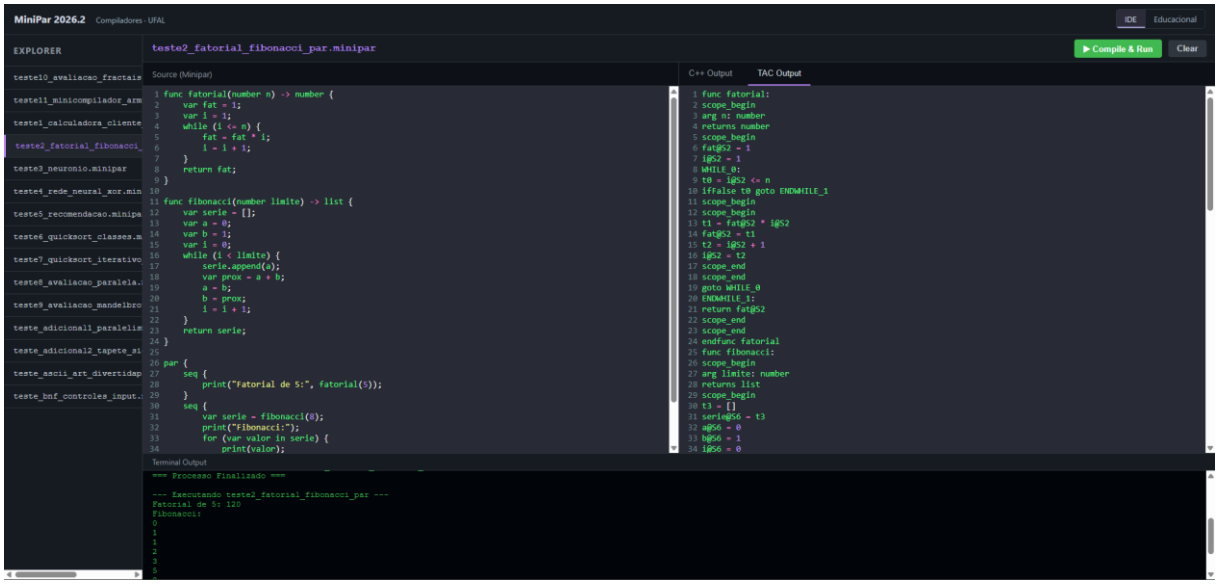
----------------------------------------
```

Fonte: captura de tela do sistema MiniPar 2026.2, elaborada pelos autores (2026).

Interface 2 — IDE Web original (/). Lista os programas, edita o código e, com "Compile & Run", mostra o C++ e o TAC gerados em abas e a saída da execução no terminal da página.

Figura 3 – IDE Web original

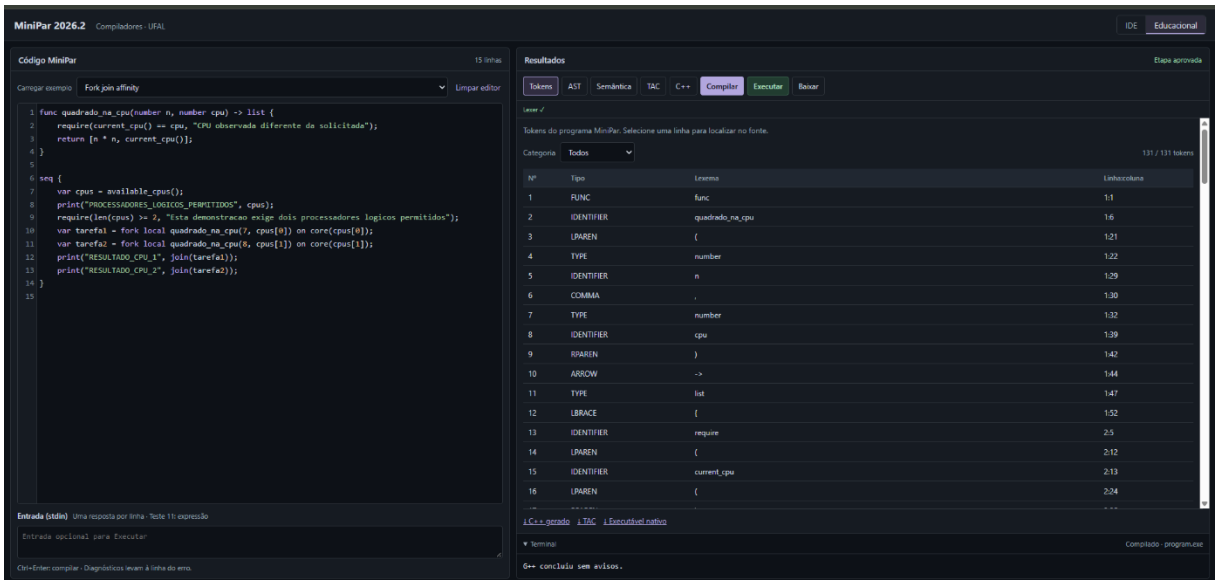




Fonte: captura de tela do sistema MiniPar 2026.2, elaborada pelos autores (2026).

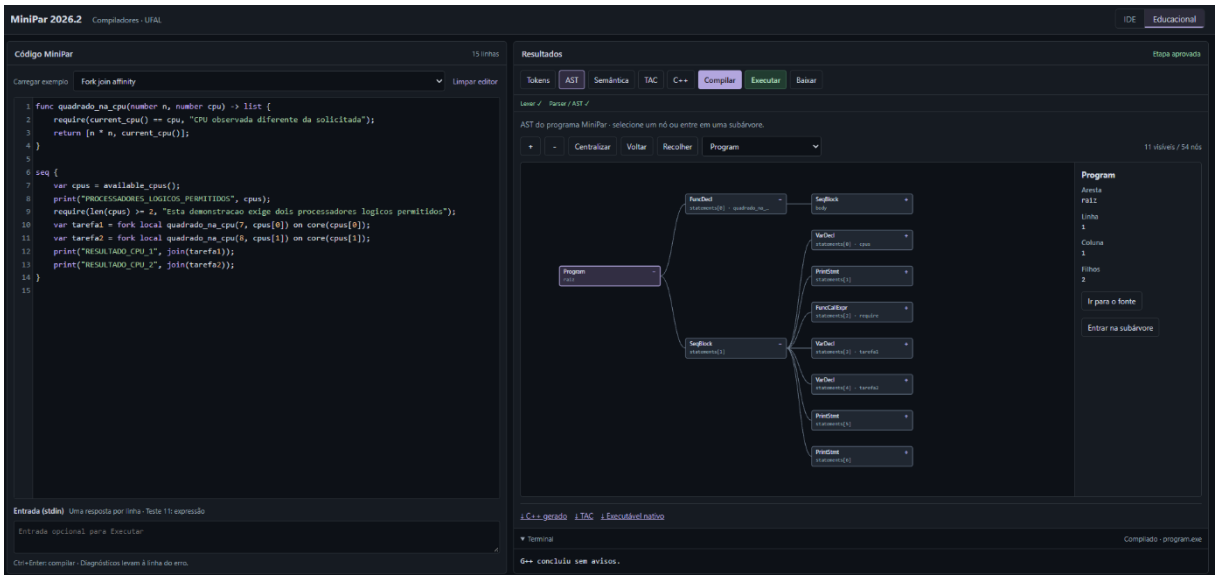
Interface 3 — Web educacional (`/educacional`). Mostra cada etapa separadamente: Tokens (tabela com tipo, lexema, linha e coluna), AST (árvore gráfica interativa com zoom, expansão e detalhes do nó), Semântica (erros e tabela de símbolos), TAC e C++, além de compilar, executar com entrada e baixar os arquivos.

Figura 4 – Web educacional: tokens



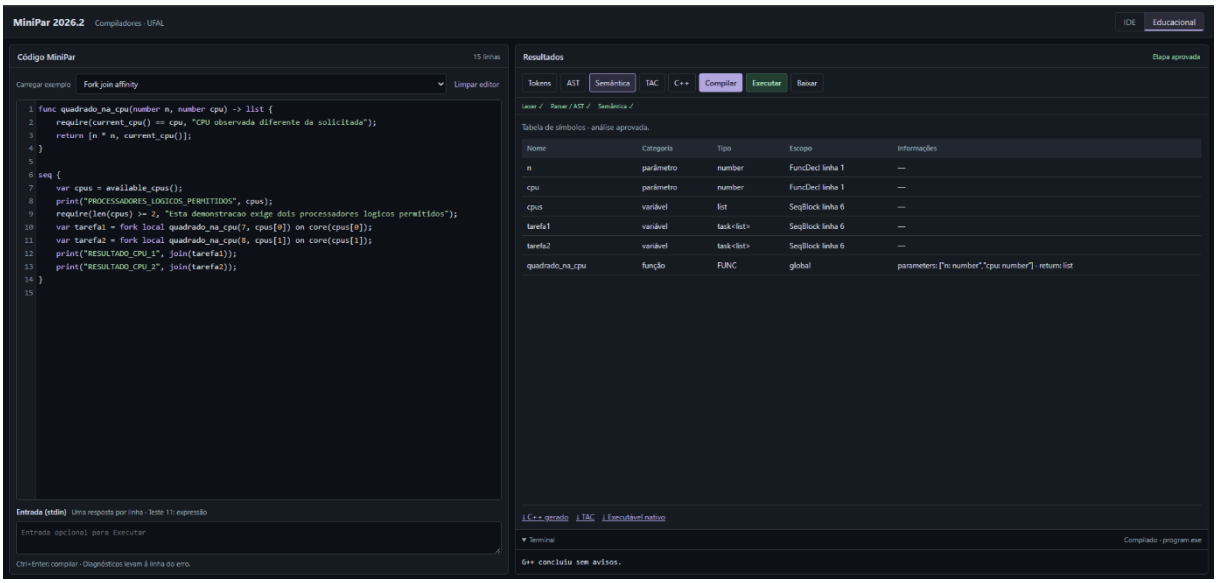
Fonte: captura de tela do sistema MiniPar 2026.2, elaborada pelos autores (2026).

Figura 5 – Web educacional: AST gráfica



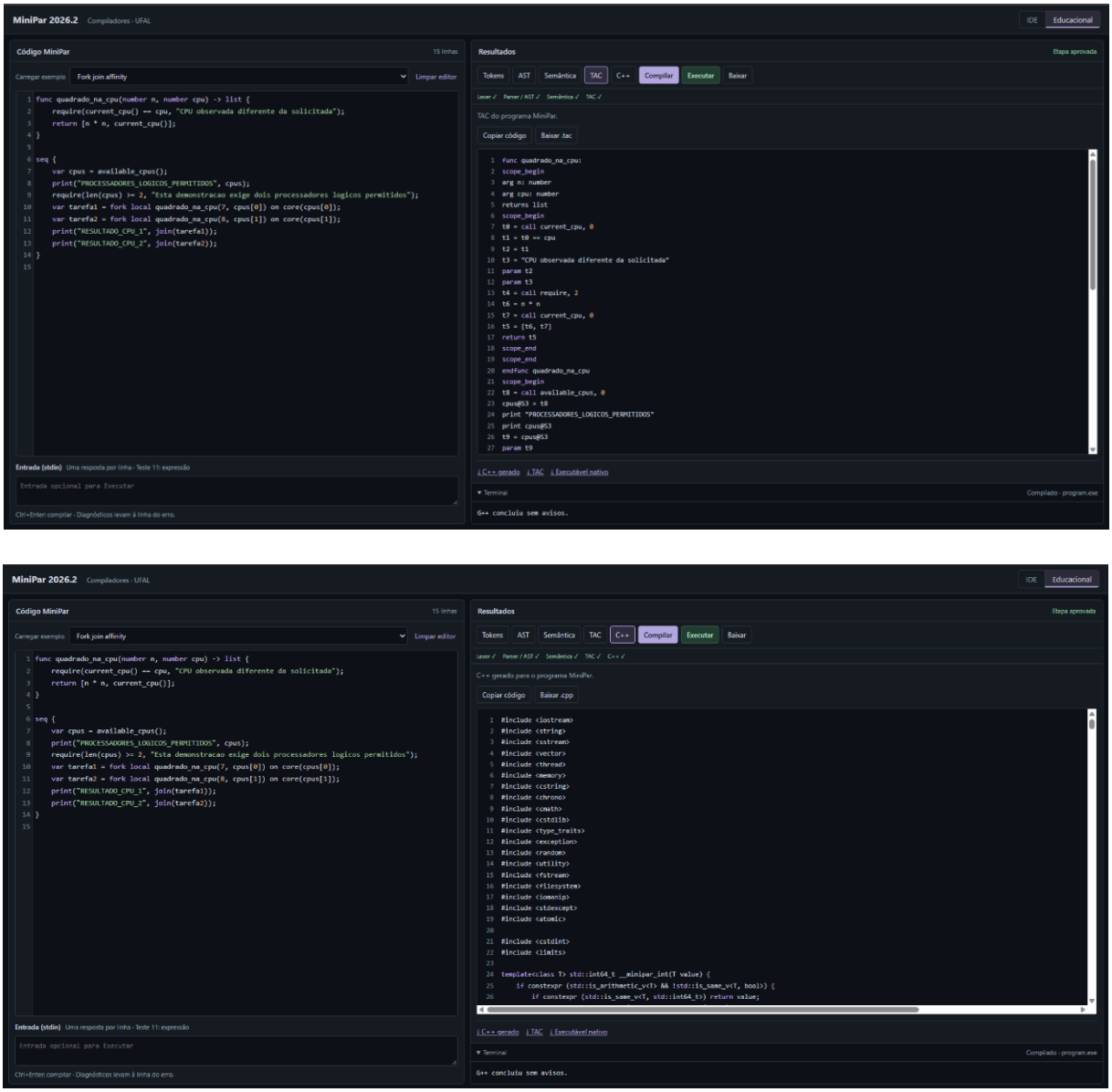
Fonte: captura de tela do sistema MiniPar 2.6.2, elaborada pelos autores (2026).

Figura 6 – Web educacional: semântica e tabela de símbolos



Fonte: captura de tela do sistema MiniPar 2.6.2, elaborada pelos autores (2026).

Figura 7 – Web educacional: TAC e C++



Fonte: captura de tela do sistema MiniPar 2026.2, elaborada pelos autores (2026).

11 PRODUCT BACKLOG E SPRINT BACKLOGS

Quadro 8 – Product Backlog

ID	Funcionalidade	Prioridade	Situação
PB01	Preservar a base 2026.1, o terminal e a IDE Web	Alta	Concluído
PB02	Lexer, Parser, AST e BNF 2026.2 com Fork/Join	Alta	Concluído
PB03	Análise semântica e tabela de símbolos	Alta	Concluído
PB04	Geração de TAC e de C++ com compilação G++ `-O3`	Alta	Concluído
PB05	Runtime de SEQ, PAR, Fork e Join com isolamento	Alta	Concluído
PB06	Canais TCP e Fork remoto (trabalhador)	Alta	Concluído em localhost; duas máquinas pendente

ID	Funcionalidade	Prioridade	Situação
PB07	Seleção de processador lógico (`on core`)	Alta	Concluído (Windows)
PB08	Programas de teste 1–7	Alta	Concluído
PB09	Programas de teste 8–10 (paralelos e fractais)	Alta	Concluído
PB10	Teste 11: minicompilador para ARMv7	Alta	Concluído
PB11	Interface Web educacional integrada	Alta	Concluído
PB12	UML, backlogs, testes de integração e relatório	Média	Concluído

Fonte: elaborado pelos autores (2026).

Quadro 9 – Sprint Backlogs

Sprint	Itens	Atividades principais
1 – Base e front-end	PB01–PB03	Ambiente G++, gramática, nós de Fork/Join, regras semânticas
2 – Geração e runtime	PB04–PB07	TAC dinâmico, C++, tarefas, snapshots, canais TCP, RPC, afinidade
3 – Programas de teste	PB08–PB10	Testes 1–11 em MiniPar, validadores, CPUlator
4 – Interfaces e documentação	PB11–PB12	Web educacional, integração das Webs, UML, relatório

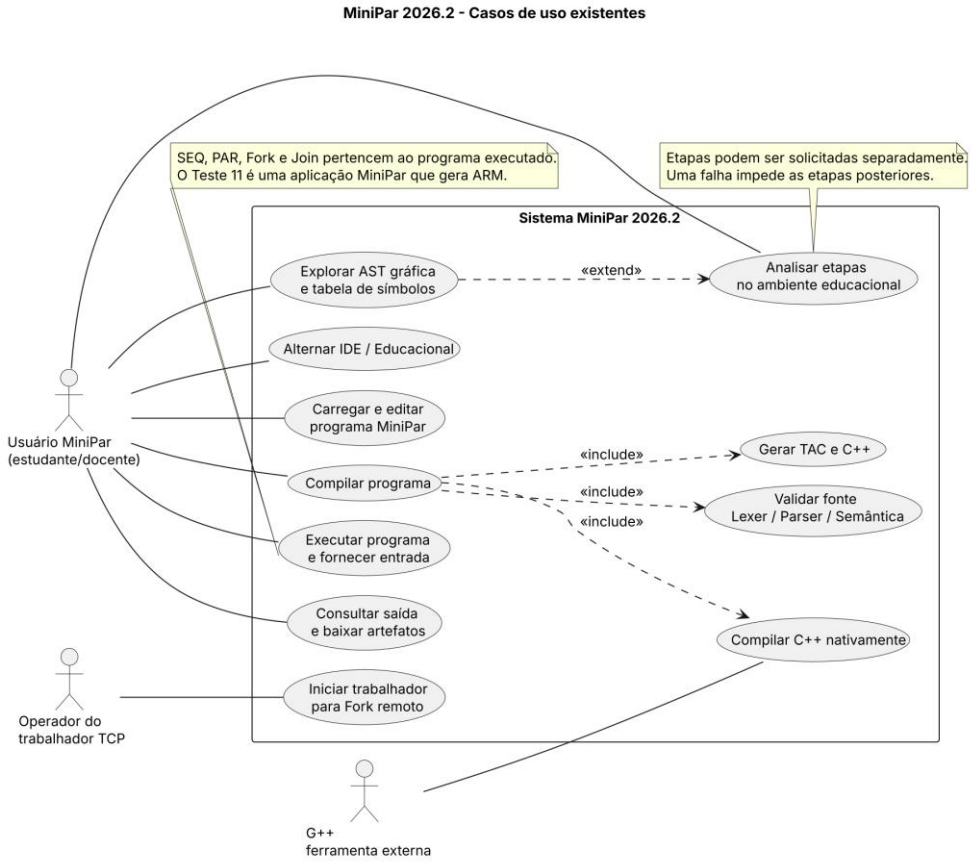
Fonte: elaborado pelos autores (2026).

As sprints organizam as etapas registradas no repositório; não foram registradas datas nem cerimônias formais.

12 MODELAGEM UML

O diagrama de casos de uso (Figura 8) mostra o usuário editando, compilando (o que inclui validar, gerar TAC/C++ e compilar com o G++), executando, consultando artefatos e explorando as etapas na Web educacional, e o operador que inicia o trabalhador para Forks remotos.

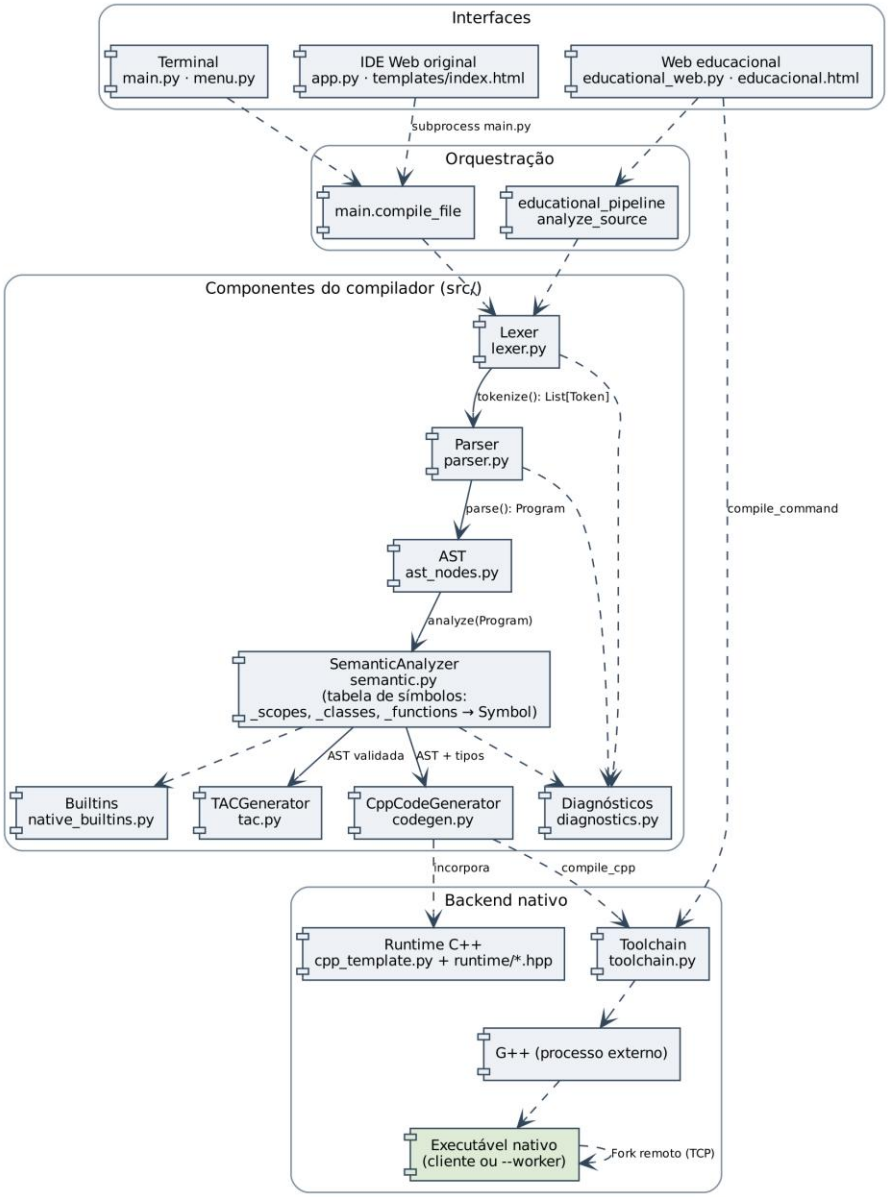
Figura 8 – Diagrama de casos de uso



Fonte: elaborado pelos autores (2026).

O diagrama de componentes (Figura 9) mostra as três interfaces, as duas fachadas de orquestração, os componentes do compilador — incluindo os geradores de TAC e de C++ pedidos no Anexo 1 do enunciado — e o backend nativo.

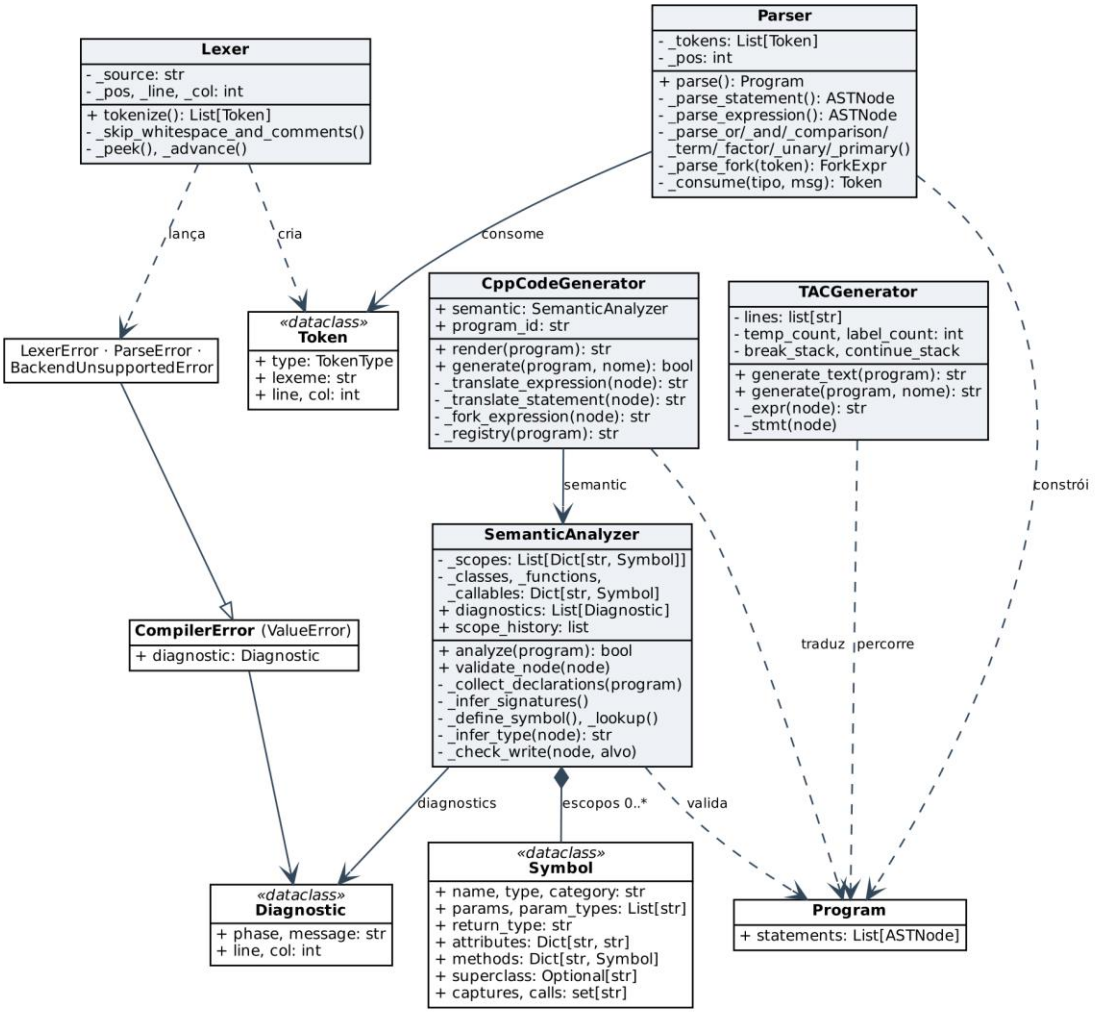
Figura 9 – Diagrama de componentes



Fonte: elaborado pelos autores (2026).

O diagrama de classes (Figura 10) mostra as classes dos componentes e suas relações.

Figura 10 – Diagrama de classes



Fonte: elaborado pelos autores (2026).

13 TESTES DE INTEGRAÇÃO

A suíte pytest (tests/unit, tests/integration e testes das interfaces) verifica a integração entre os componentes, compilando e executando código real. O registro da última validação consolidada informa 519 testes aprovados, sem falhas (Windows 11, Python 3.14.3, G++ 16.2.0).

Quadro 10 – Integrações verificadas

Integração	**Arquivos de teste**	**O que é verificado**
Lexer → Parser → AST	`test_frontend.py`, `test_tasks_grammar.py`	Tokens, precedência, nós de Fork/Join, erros de sintaxe
AST → Semântica → Símbolos	`test_semantic_2026_2.py`, `test_types_1a7.py`	Escopos, tipos, chamadas, herança, isolamento
AST → TAC e C++	`test_tac_2026_2.py`, `test_generation.py`	TAC dinâmico e C++ gerados da mesma AST

Integração	Arquivos de teste	O que é verificado
C++ → G++ → executável	`test_native.py`, `test_programs_1a7.py`	Compilação real com `-O3` e execução
Runtime, TCP e trabalhador	`test_runtime.py`, `test_affinity_native.py`	Threads, canais, Fork remoto, afinidade, erros
Interfaces Web	`test_web.py`, `test_educational_web.py`	APIs Flask, etapas educacionais e execução

Fonte: elaborado pelos autores (2026).

14 TESTES DOS PROGRAMAS DE EXEMPLO

Os programas estão em `tests/` e foram compilados e executados pelos validadores `tools/validate_programs.py` (1–7) e `tools/validate_parallel.py` (8–10), com todas as fases aprovadas. Os resultados abaixo são os registrados nessas execuções.

14.1 Teste 1 — Calculadora cliente/servidor

Uma tarefa servidor e uma tarefa cliente trocam mensagens por dois canais TCP (portas 5101 e 5102). O cliente lê a operação e os operandos com `input` e os envia; o servidor calcula com a função `calcular` e devolve o resultado, tratando divisão por zero. Com `+`, `2` e `3`, a saída foi “Resultado: 5.000000”. O teste foi feito em localhost.

Figura 11 – Teste 1

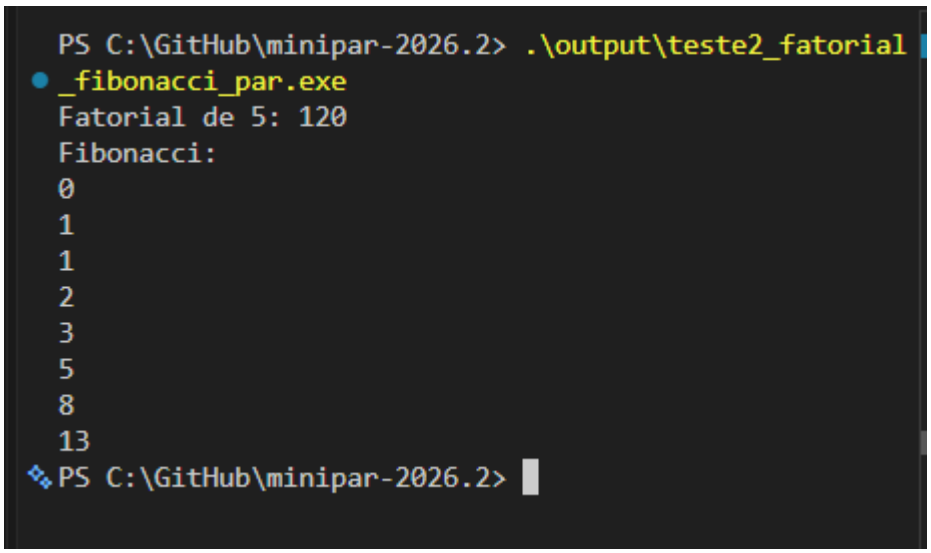


Fonte: captura de tela do sistema MiniPar 2026.2, elaborada pelos autores (2026).

14.2 Teste 2 — Fatorial e Fibonacci em paralelo

Um bloco `par` executa `fatorial(5)` e `fibonacci(8)` em duas threads. Saída: “Fatorial de 5: 120” e a série 0, 1, 1, 2, 3, 5, 8, 13.

Figura 12 – Teste 2

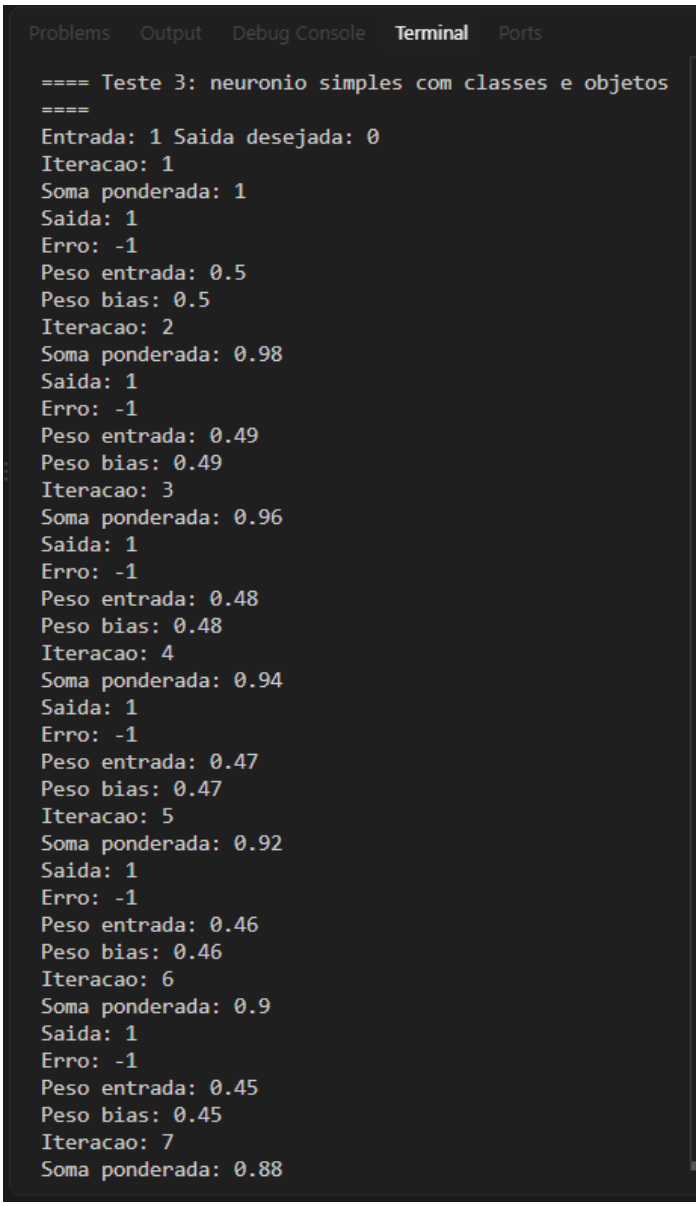


Fonte: captura de tela do sistema MiniPar 2026.2, elaborada pelos autores (2026).

14.3 Teste 3 — Neurônio

A classe Neuronio, com construtor init, implementa o treinamento de um neurônio simples por meio da atualização iterativa de seus pesos. A Figura 13 apresenta as primeiras etapas da execução, evidenciando o cálculo da soma ponderada, a saída produzida, o erro e a atualização dos pesos. O registro do projeto indica convergência na iteração 51, embora a captura apresentada documente apenas o início do treinamento.

Figura 13 – Teste 3



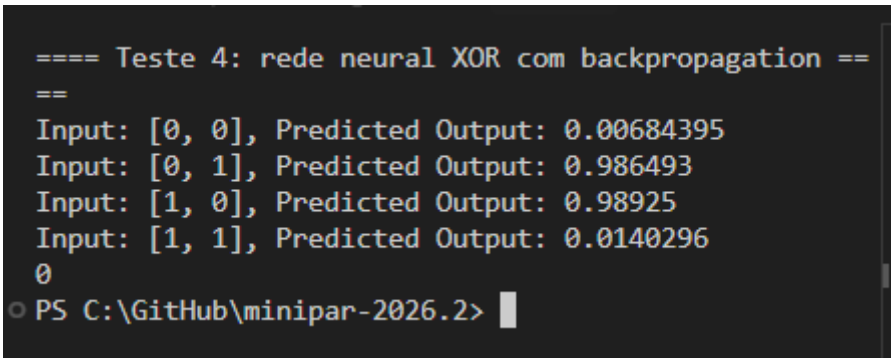
A imagem apresenta um trecho das iterações do treinamento, evidenciando a atualização dos pesos e o cálculo do erro. Devido à extensão da saída produzida pelo programa, foram selecionadas apenas algumas etapas representativas de sua execução.

Fonte: captura de tela do sistema MiniPar 2026.2, elaborada pelos autores (2026).

14.4 Teste 4 — Rede neural XOR

A classe `RedeNeuralXOR` implementa uma rede 2–3–1 com sigmoide, feedforward e backpropagation, treinada por 20.000 épocas (taxa 0,5, pesos iniciais fixos). Previsões: 0,0068 para [0,0], 0,9865 para [0,1], 0,9893 para [1,0] e 0,0140 para [1,1].

Figura 14 – Teste 4

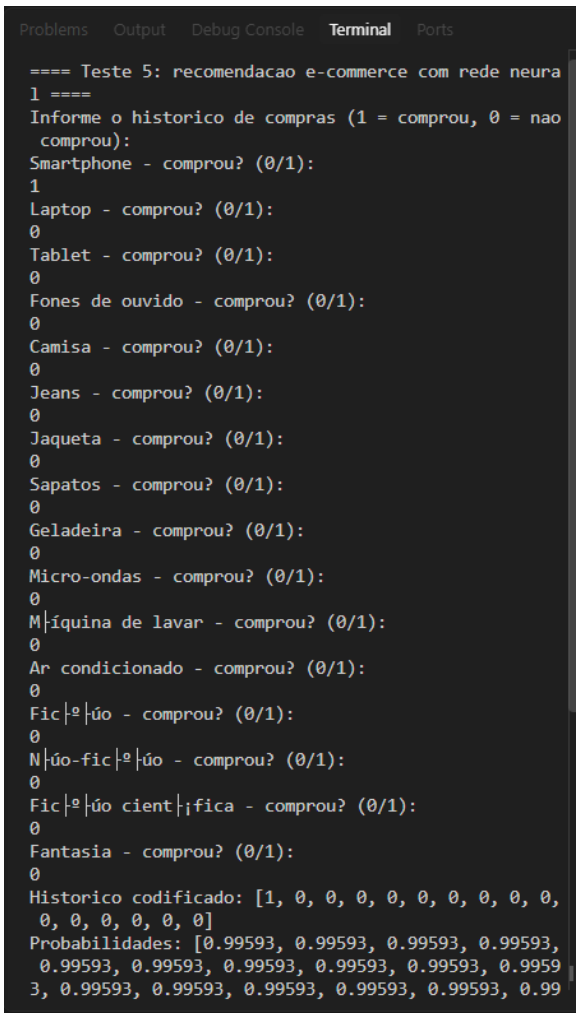


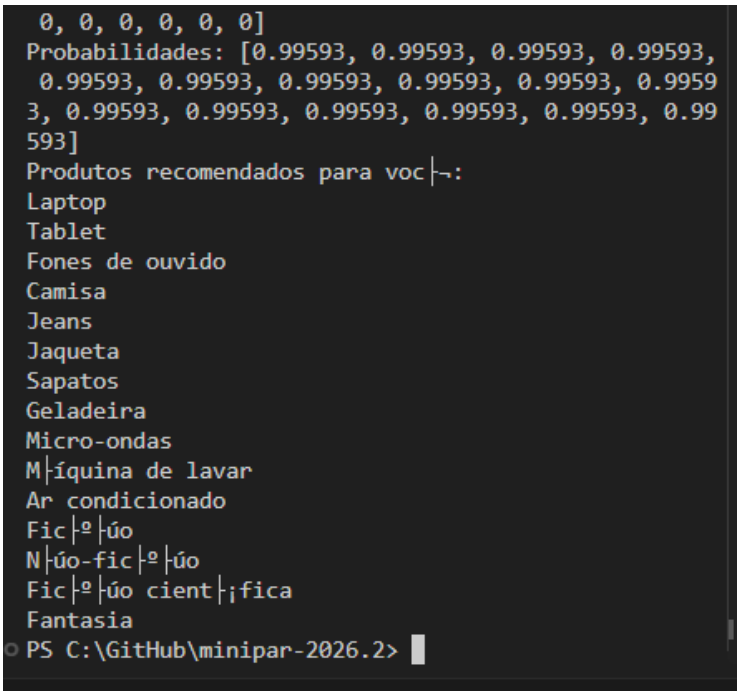
Fonte: captura de tela do sistema MiniPar 2026.2, elaborada pelos autores (2026).

14.5 Teste 5 — Sistema de recomendação

Cinco classes (Produto, Categoria, Usuario, RedeNeural e Recomendador) codificam o histórico de compras e aplicam uma rede neural 16–10–16, utilizando as funções de ativação ReLU e sigmoide, com pesos inicializados em 0,5, conforme o modelo proposto no enunciado. Na execução documentada, o usuário informou a compra de um produto entre os 16 disponíveis. Após o processamento do histórico, o sistema apresentou os 15 produtos restantes como recomendações, acompanhados das probabilidades calculadas pela rede neural.

Figura 15 – Teste 5



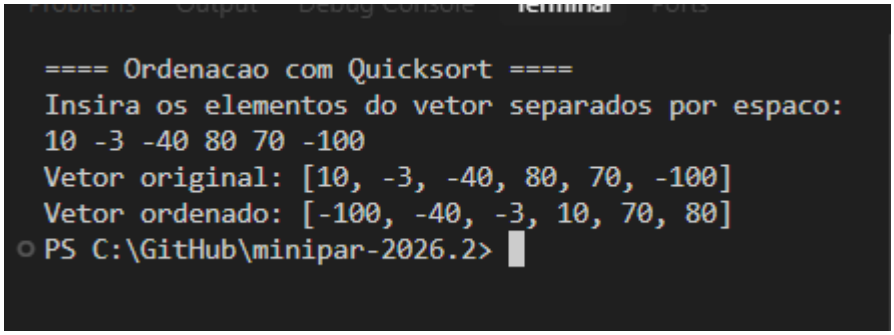


Fonte: captura de tela do sistema MiniPar 2026.2, elaborada pelos autores (2026).

14.6 Teste 6 — Quicksort recursivo com classes

A classe `Quicksort` ordena com os métodos recursivos `quicksort` e `particionar`. Entrada `10 -3 -40 80 70 -100`; saída `[-100, -40, -3, 10, 70, 80]`.

Figura 16 – Teste 6

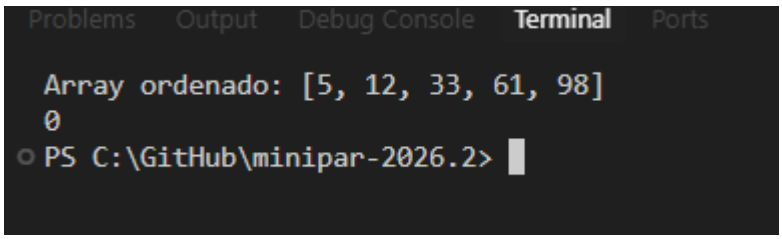


Fonte: captura de tela do sistema MiniPar 2026.2, elaborada pelos autores (2026).

14.7 Teste 7 — Quicksort iterativo

A função `quicksort_iterativo` usa uma lista como pilha de intervalos, sem recursão. `[33, 12, 98, 5, 61]` resultou em `[5, 12, 33, 61, 98]`. O TAC gerado está no Capítulo 8.

Figura 17 – Teste 7



Fonte: captura de tela do sistema MiniPar 2026.2, elaborada pelos autores (2026).

14.8 Testes 8, 9 e 10 — Avaliação de atividades e fractais

Os três programas avaliam 160 atividades (40 alunos × 4 atividades) primeiro de forma sequencial e depois em paralelo, dividindo-as entre quatro avaliadores (Professor e três Monitores) com `fork local` e recolhendo os resultados com `join`. O Teste 9 acrescenta uma quinta tarefa, que gera o Mandelbrot em ASCII (50 × 20), e o Teste 10 acrescenta uma sexta, que gera o tapete de Sierpinski de grau 4 em SVG. Uma barreira com canais garante que todas as tarefas estejam ativas antes de começar. Todos os programas verificaram as 160 atividades

sem perdas nem duplicatas e fractais iguais nos dois modos. Em uma execução com `on core`, as tarefas rodaram nos processadores lógicos 0–3, 0–4 e 0–5, como solicitado.

Quadro 11 – Tempos medidos nos Testes 8, 9 e 10

Teste	Tarefas	Sequencial (s)	Paralelo (s)	Speedup
8	4	0,0222	0,0195	1,143
9	5	0,0221	0,0453	0,488
10	6	0,0277	0,5576	0,050

Fonte: elaborado pelos autores (2026).

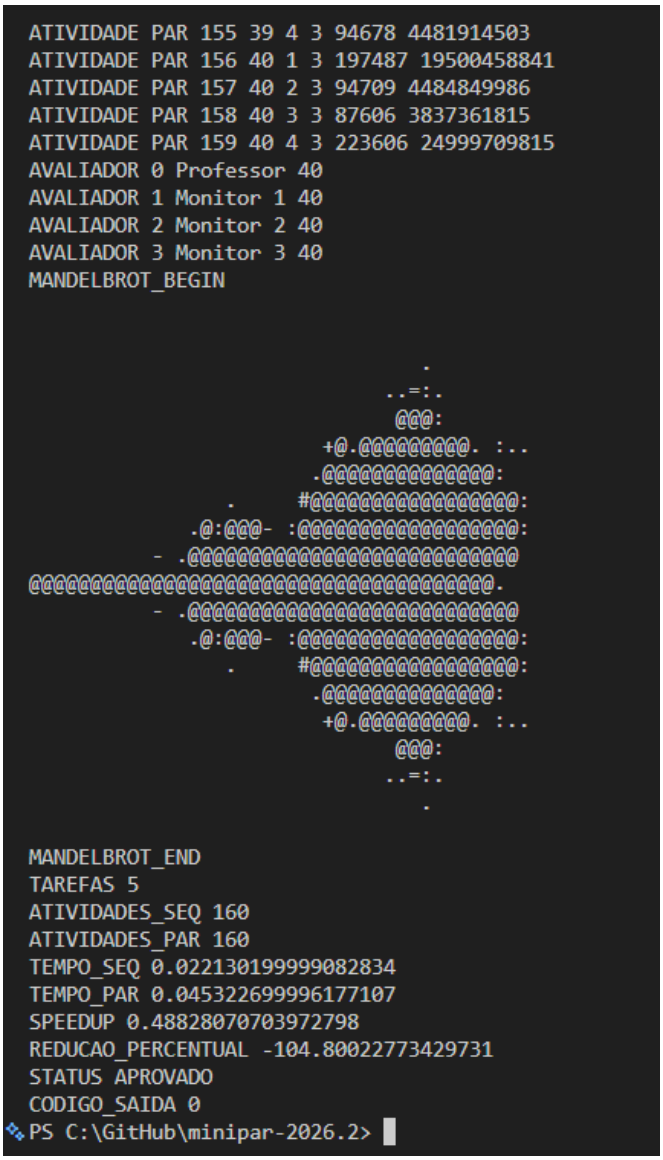
Nas execuções registradas, o Teste 8 apresentou uma pequena aceleração na modalidade paralela, enquanto os Testes 9 e 10 registraram tempos superiores aos da execução sequencial. Essa diferença está relacionada aos custos de criação, coordenação e sincronização das tarefas, que podem superar o tempo necessário ao processamento da carga computacional. Os resultados variam conforme as condições de execução e não representam desempenho garantido em todos os ambientes.

Figura 18 – Teste 8

```
PS C:\GitHub\minipar-2026.2> .\output\teste8_avalia
alela.exe | Select-Object -Last 45
>> $LASTEXITCODE
ATIVIDADE PAR 139 35 4 3 148974 11096551851
ATIVIDADE PAR 140 36 1 3 154193 11887663528
ATIVIDADE PAR 141 36 2 3 207778 21585744753
ATIVIDADE PAR 142 36 3 3 91920 4224597240
ATIVIDADE PAR 143 36 4 3 194445 18904331790
ATIVIDADE PAR 144 37 1 3 175838 15459413203
ATIVIDADE PAR 145 37 2 3 85458 3651492153
ATIVIDADE PAR 146 37 3 3 221066 24434977645
ATIVIDADE PAR 147 37 4 3 101291 5129882695
ATIVIDADE PAR 148 38 1 3 125863 7920684453
ATIVIDADE PAR 149 38 2 3 129526 8388427575
ATIVIDADE PAR 150 38 3 3 118605 7033513710
ATIVIDADE PAR 151 38 4 3 100280 5027989060
ATIVIDADE PAR 152 39 1 3 223249 24919946376
ATIVIDADE PAR 153 39 2 3 144263 10405834453
ATIVIDADE PAR 154 39 3 3 88154 3885519781
ATIVIDADE PAR 155 39 4 3 94678 4481914503
ATIVIDADE PAR 156 40 1 3 197487 19500458841
ATIVIDADE PAR 157 40 2 3 94709 4484849986
ATIVIDADE PAR 158 40 3 3 87606 3837361815
ATIVIDADE PAR 159 40 4 3 223606 24999709815
AVALIADOR 0 Professor 40
AVALIADOR 1 Monitor 1 40
AVALIADOR 2 Monitor 2 40
AVALIADOR 3 Monitor 3 40
TAREFAS 4
ATIVIDADES_SEQ 160
ATIVIDADES_PAR 160
TEMPO_SEQ 0.022236400021938607
TEMPO_PAR 0.01946090001729317
SPEEDUP 1.1426193034329912
REDUCAO_PERCENTUAL 12.481786628712863
STATUS APROVADO
0
PS C:\GitHub\minipar-2026.2>
```

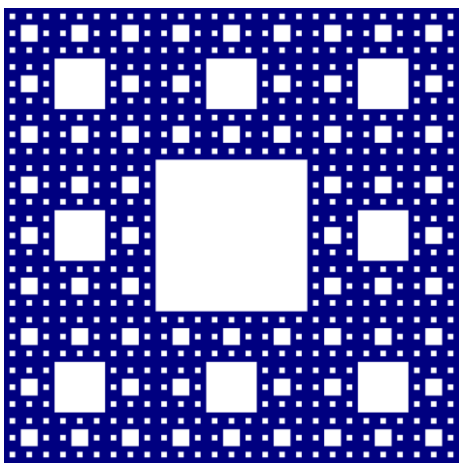
Fonte: captura de tela do sistema MiniPar 2026.2, elaborada pelos autores (2026).

Figura 19 – Teste 9



Fonte: captura de tela do sistema MiniPar 2026.2, elaborada pelos autores (2026).

Figura 20 – Tapete de Sierpinski gerado pelo Teste 10 (SVG)



Fonte: arquivo `teste10-sierpinski.svg` gerado pelo executável do Teste 10.

Figura 21 – Teste 10



Fonte: captura de tela do sistema MiniPar 2026.2, elaborada pelos autores (2026).

14.9 Teste 11 — Minicompilador de expressões para ARMv7

Programa MiniPar com onze classes (Lexer, Parser, ASTNode, SymbolTable, TACGenerator, ExpressionEvaluator, ARMCodeGenerator, MiniCompiler etc.) que, para uma expressão infixada, mostra tokens, AST, tabela de símbolos e TAC, calcula o resultado e grava um arquivo .arm, que deixa o resultado em r0. Os resultados registrados para as três expressões exigidas são:

Quadro 12 – Resultados do Teste 11

Expressão	**Resultado**	**CPUlator (ARMv7 DE1-SoC)**
`((2+4)*7)-(5*4)`	22	r0 = 0x00000016
`((10-8*7)+4)*2`	−84	r0 = 0xFFFFFFAC
`((2+4)*7)-(5*4)`	22	r0 = 0x00000016

Fonte: elaborado pelos autores (2026).

As três expressões foram verificadas manualmente no CPUlator ARMv7 DE1-SoC, conforme a Figura 23, confirmando os resultados 22, −84 e 22 no registrador r0. Adicionalmente, os testes automatizados utilizam Zig/LLVM e o emulador Unicorn para validar os arquivos Assembly gerados.

Figura 22 – Teste 11: execução e Assembly ARMv7

```
Expressao infixada: ((2+4)*7)-(5*4)
TOKENS_BEGIN
TOKEN LPAREN ( 1 1
TOKEN LPAREN ( 1 2
TOKEN NUMBER 2 1 3
TOKEN ADD + 1 4
TOKEN NUMBER 4 1 5
TOKEN RPAREN ) 1 6
TOKEN MUL * 1 7
TOKEN NUMBER 7 1 8
TOKEN RPAREN ) 1 9
TOKEN SUB - 1 10
TOKEN LPAREN ( 1 11
TOKEN NUMBER 5 1 12
TOKEN MUL * 1 13
TOKEN NUMBER 4 1 14
TOKEN RPAREN ) 1 15
TOKENS_END
AST_BEGIN
ROOT BINARY -
  L BINARY *
    L BINARY +
      L NUMBER 2
      R NUMBER 4
    R NUMBER 7
  R BINARY *
    L NUMBER 5
    R NUMBER 4

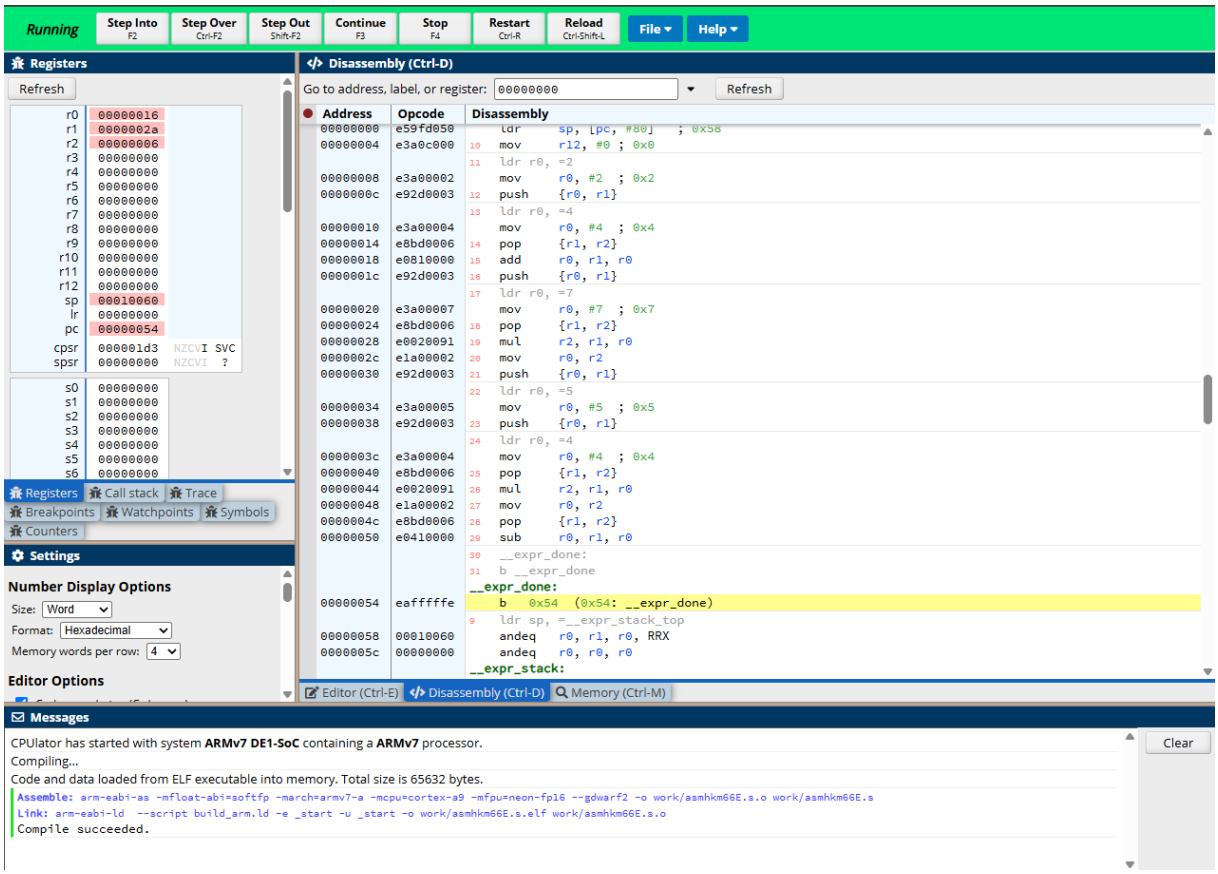
AST_END
SIMBOLOS_BEGIN
SIMBOLO 2 CONST 2
SIMBOLO 4 CONST 4
SIMBOLO 7 CONST 7
SIMBOLO 5 CONST 5
SIMBOLOS_END
TAC_BEGIN
t1 = 2 + 4
t2 = t1 * 7
t3 = 5 * 4
t4 = t2 - t3
```

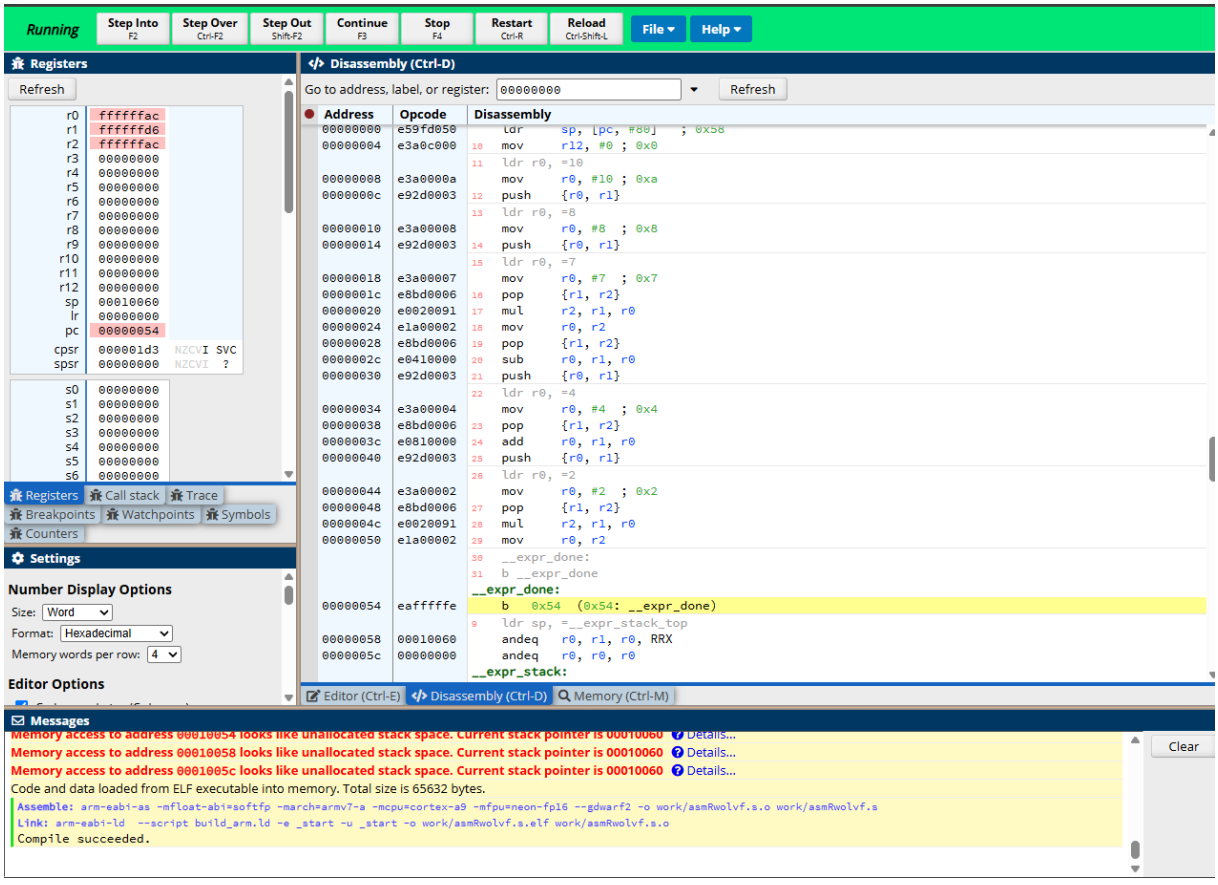
```
TAC_END
RESULTADO 22
ARM_BEGIN
.syntax unified
.arch armv7-a
.arm
.global _start
.global __expr_done
.text
.balign 4
_start:
    ldr sp, =__expr_stack_top
    mov r12, #0
    ldr r0, =2
    push {r0, r1}
    ldr r0, =4
    pop {r1, r2}
    add r0, r1, r0
    push {r0, r1}
    ldr r0, =7
    pop {r1, r2}
    mul r2, r1, r0
    mov r0, r2
    push {r0, r1}
    ldr r0, =5
    push {r0, r1}
    ldr r0, =4
    pop {r1, r2}
    mul r2, r1, r0
    mov r0, r2
    pop {r1, r2}
    sub r0, r1, r0
__expr_done:
    b __expr_done
.ltorg
.data
.balign 4
.bss
.balign 8
__expr_stack:
.space 65536
__expr_stack_top:

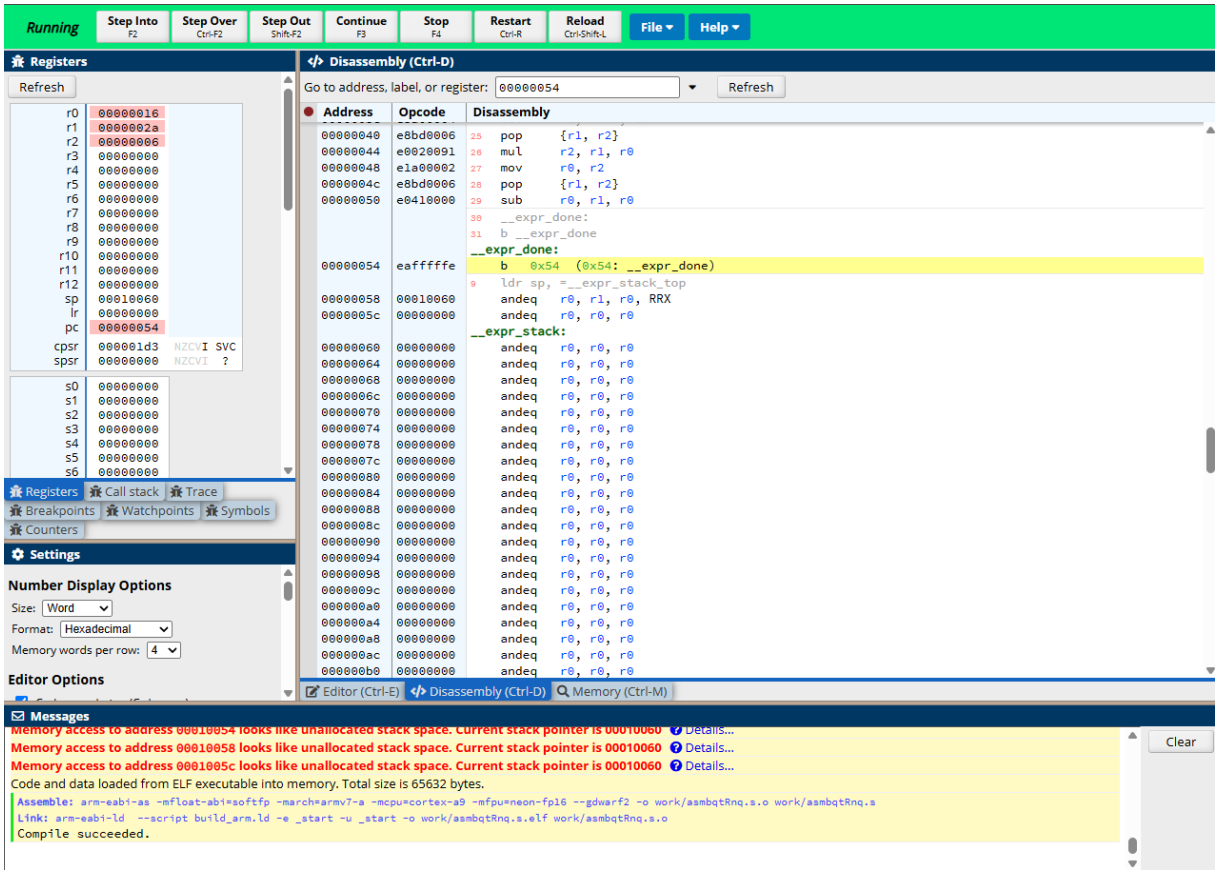
ARM_END
ARQUIVO: output/teste11-expressao.arm
STATUS APROVADO
PS C:\GitHub\minipar-2026.2>
```

Fonte: captura de tela do sistema MiniPar 2026.2, elaborada pelos autores (2026).

Figura 23 – Teste 11 no CPUlator







Fonte: captura de tela do sistema MiniPar 2026.2, elaborada pelos autores (2026).

15 IMPLEMENTAÇÃO

Tecnologias e ambiente: Python 3.14 (compilador e interfaces); C++20 e G++ 16.2 do MSYS2 (código gerado); Flask, HTML, CSS, JavaScript e CodeMirror (interfaces Web); pytest (testes); Zig, Unicorn e CPUlator (Teste 11); PlantUML e Graphviz (UML); Git/GitHub; Windows 11. O código-fonte tem comentários explicativos nos módulos de `src/`.

Como executar:

```
pip install -r requirements-dev.lock.txt
python main.py                                          # terminal ou Web
python app.py                                           # http://127.0.0.1:5000 e /educacional
python main.py tests/teste8_avaliacao_paralela.minipar   # gera .tac, .cpp e executável
python -m pytest                                        # suíte de testes
```

As telas de execução de cada programa estão nos Capítulos 9, 10 e 14.

16 CONCLUSÃO

O compilador MiniPar 2026.2 implementa as principais etapas previstas no projeto: análise léxica, sintática e semântica, tabela de símbolos, geração de TAC, tradução para C++20 e compilação nativa com otimização pelo G++. A linguagem também oferece orientação a objetos, blocos SEQ e PAR, Fork e Join, seleção de processadores lógicos e comunicação por canais TCP.

Os onze programas obrigatórios foram implementados, e os registros de validação informam 519 testes automatizados aprovados. As execuções documentadas demonstram o funcionamento das interfaces, dos algoritmos e da geração de Assembly ARMv7, incluindo os resultados obtidos no CPUlator.

Nos testes de desempenho, a execução paralela apresentou uma pequena aceleração no Teste 8, mas foi mais lenta nos Testes 9 e 10, em razão dos custos adicionais de coordenação e sincronização. A comunicação distribuída foi validada entre processos por TCP no mesmo computador, permanecendo pendente sua demonstração em dois computadores físicos.

REFERÊNCIAS

AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. **Compiladores**: princípios, técnicas e ferramentas. 2. ed. São Paulo: Pearson Addison-Wesley, 2008.

DOMINGUEZ, Arturo Hernandez. **Projeto do Compilador MiniPar 2026.2**. Maceió: Instituto de Computação, UFAL, 2026. Material da disciplina de Compiladores.

SOMMERVILLE, Ian. **Software engineering**. 10. ed. Boston: Pearson, 2016.

WILLIAMS, Anthony. **C++ concurrency in action**. 2. ed. Shelter Island: Manning, 2019.

FREE SOFTWARE FOUNDATION. **GCC online documentation**. Disponível em: https://gcc.gnu.org/onlinedocs/.

PALLETS PROJECTS. **Flask documentation**. Disponível em: https://flask.palletsprojects.com/.